Block Cipher Modes of Operation

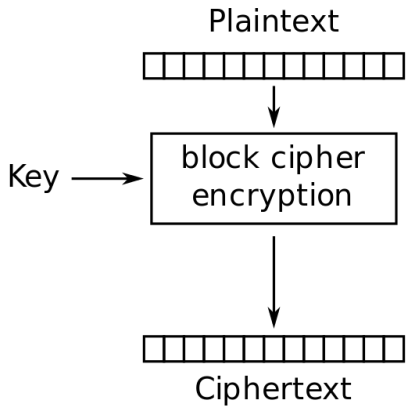


Textbook Chapter 6.6–6.9

Scratchpad: Let's design it together

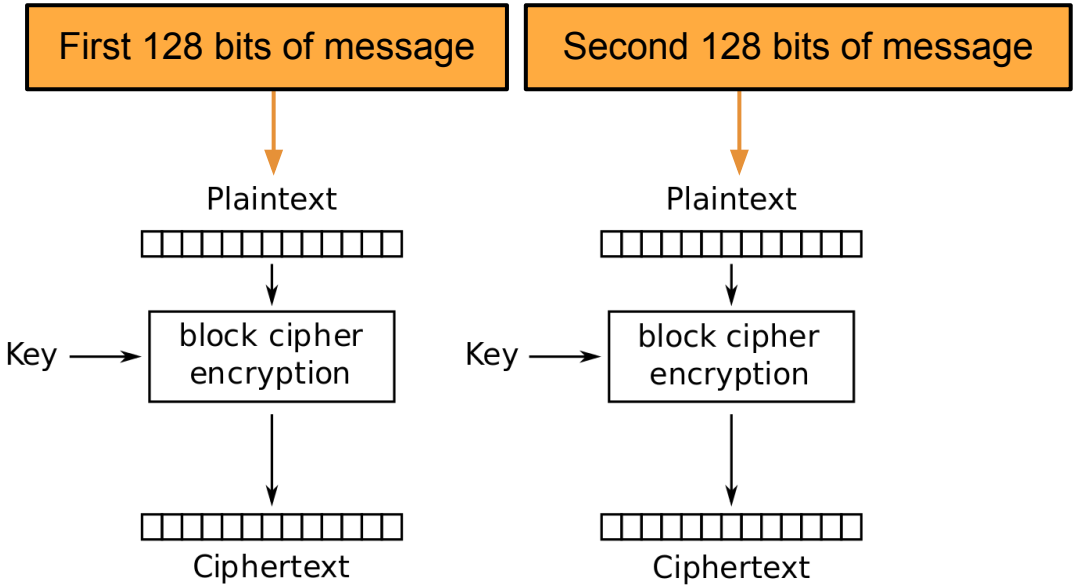
Here's an AES block. Remember that it can only encrypt 128-bit messages.

How can we use AES to encrypt a longer message (say, 256 bits?)

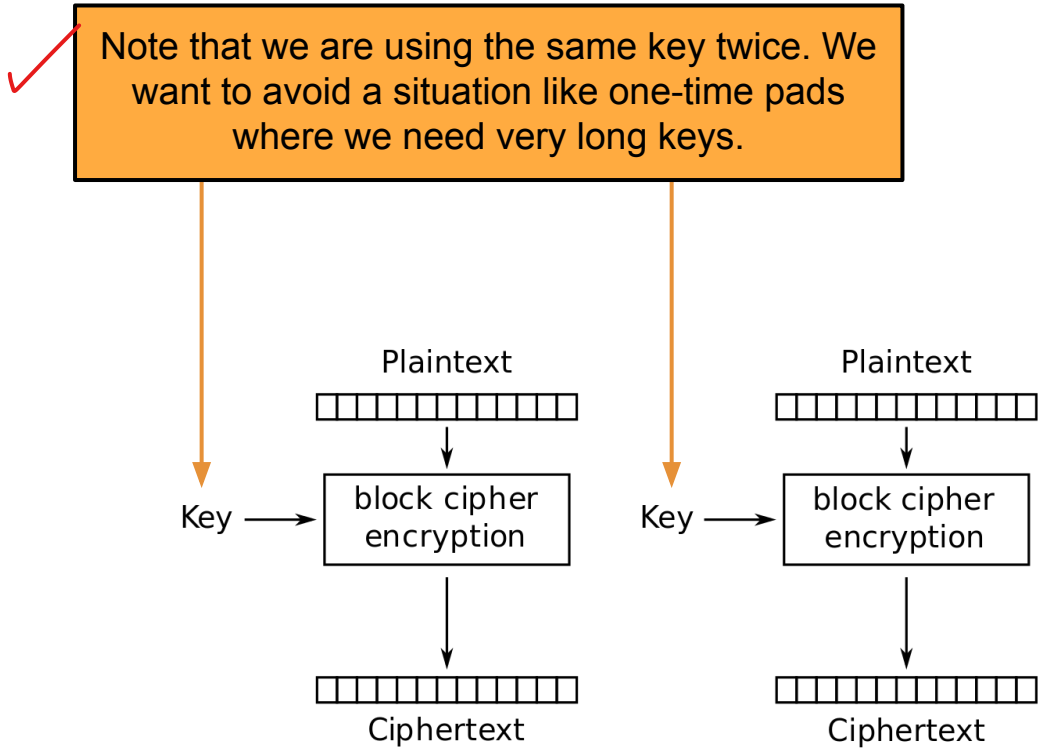


Scratchpad: Let's design it together

Idea: Let's use AES twice!

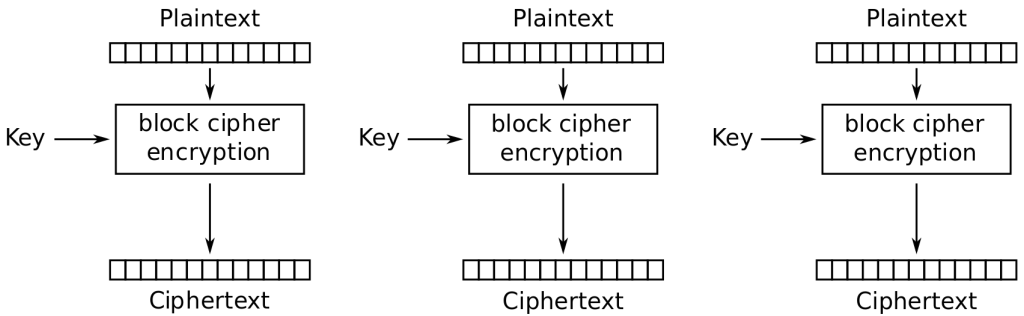


Scratchpad: Let's design it together



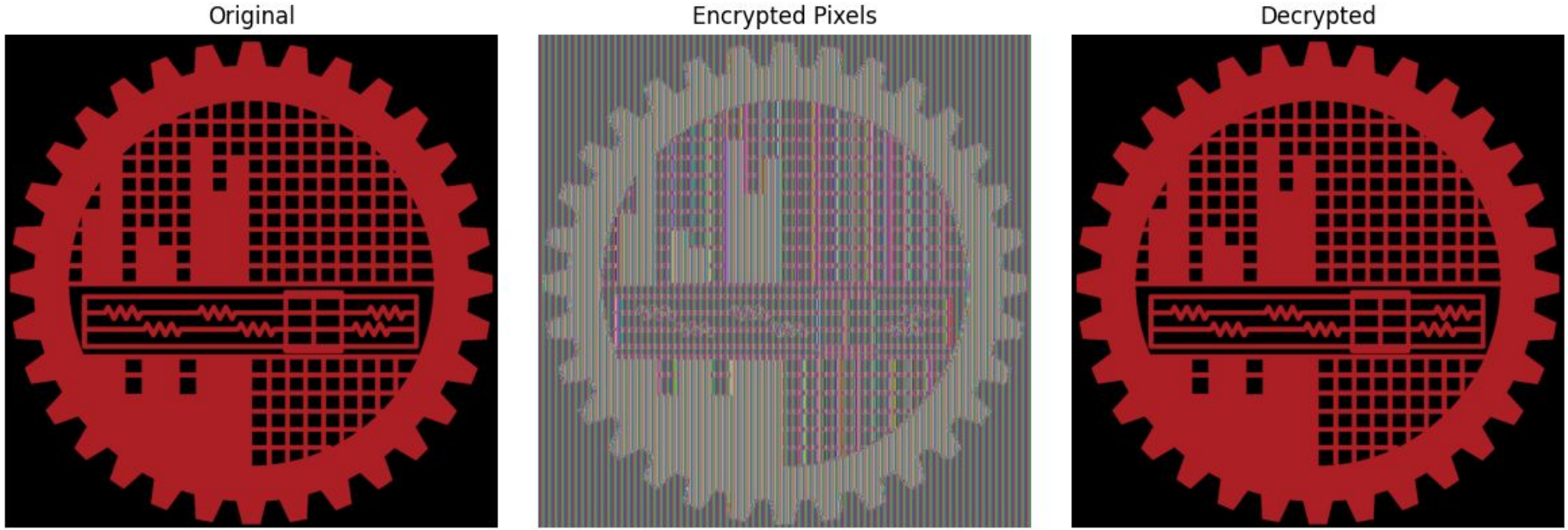
ECB Mode

- We've just designed **electronic code book (ECB) mode**
  - Enc(K, M) = C_1 || C_2 || … || C_m
  - Assume m is the number of blocks of plaintext in M, each of size n
- AES-ECB is not IND-CPA secure. Why?
  - Because ECB is deterministic.



Electronic Codebook (ECB) mode encryption

ECB Mode: Not IND-CPA Secure

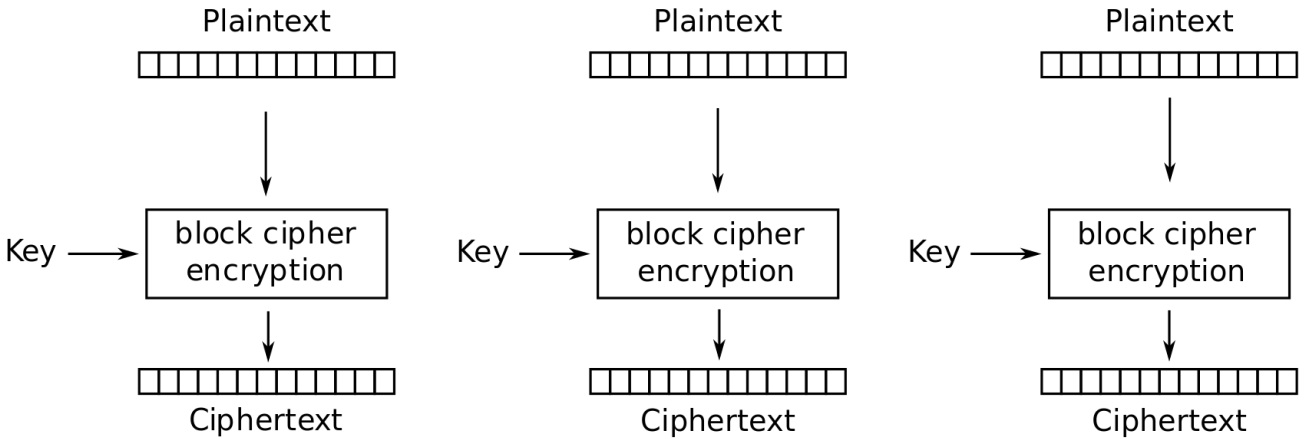


https://colab.research.google.com/drive/1JN-P9Hyrpnv1ildiEX71qbGEXvHnBQ-9?usp=sharing

Scratchpad: Let's design it together

Here's ECB mode. It's not IND-CPA secure because it's deterministic.

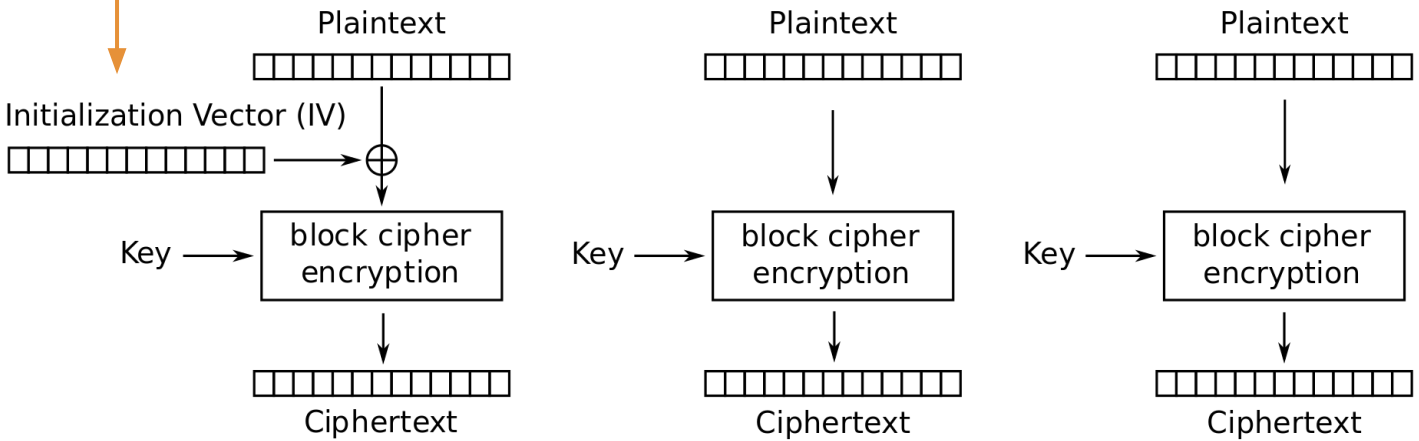
Let's fix that by adding some randomness.



Scratchpad: Let's design it together

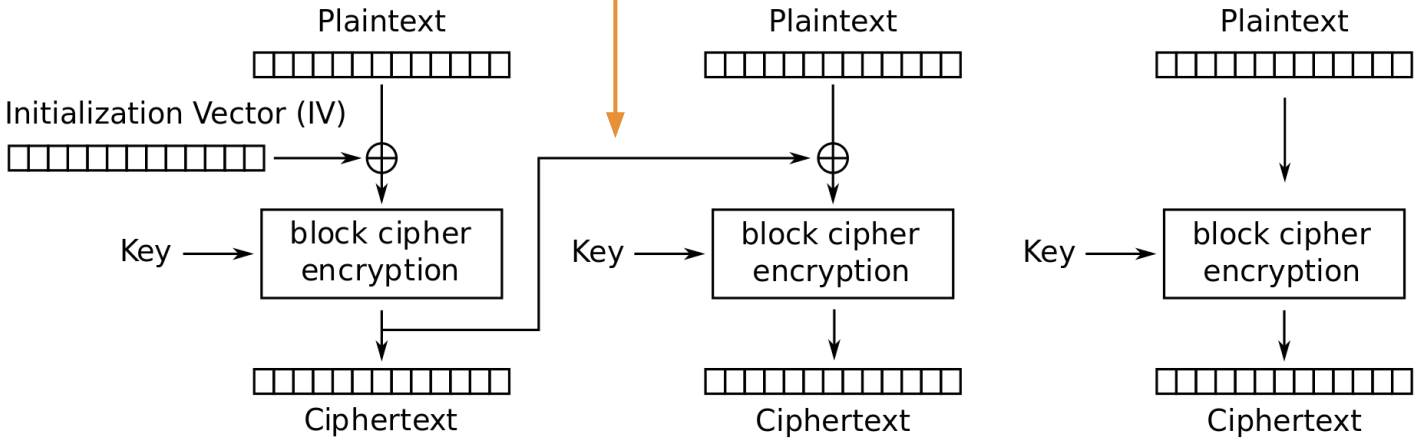
The **Initialization Vector** (**IV**) is different for every encryption. Now the first ciphertext block will be different for every encryption!

Okay, but the other blocks are still deterministic...

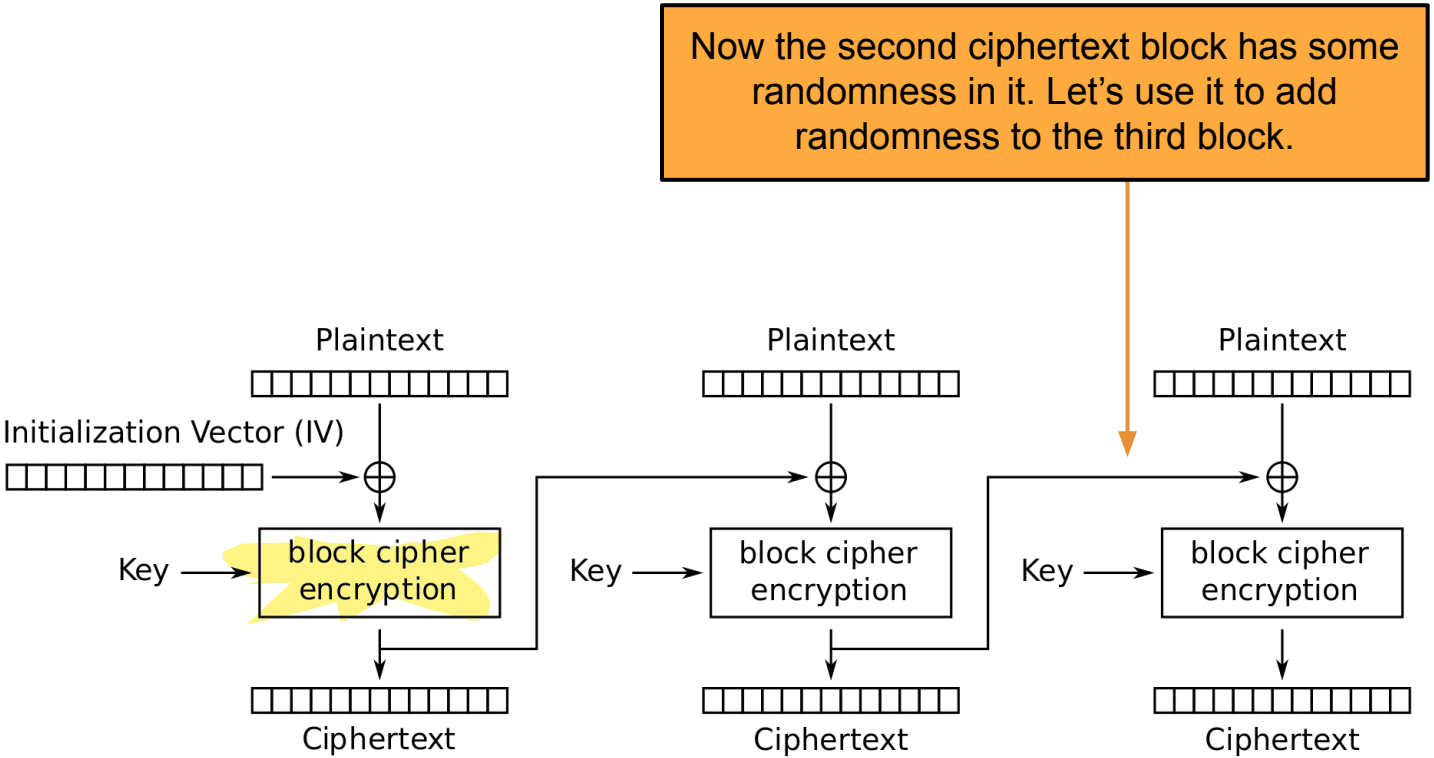


Scratchpad: Let's design it together

Idea: The first ciphertext block was computed with some randomness. Let's use it to add randomness to the second block.

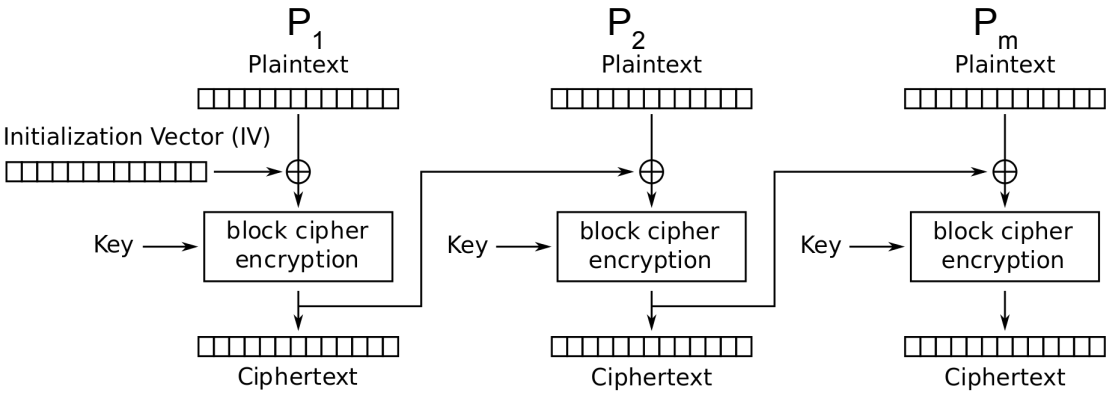


Scratchpad: Let's design it together



CBC Mode

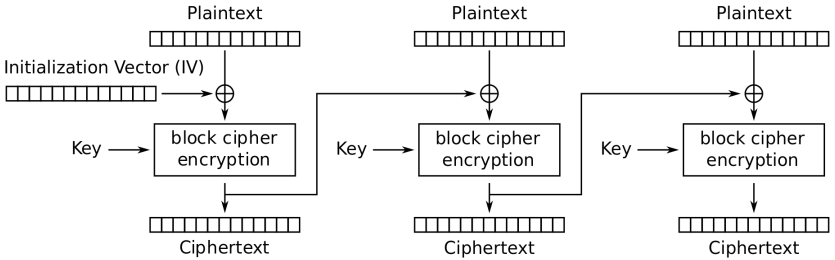
- We've just designed **cipher block chaining (CBC) mode**
- $C_i = E_K(M_i \oplus C_{i-1})$; C_0 = IV
- Enc(K, M):
  - Split M in m plaintext blocks $P_1 \ldots P_m$ each of size n
  - Choose a random IV
  - Compute and output (IV, C_1, …, C_m) as the overall ciphertext
- How do we decrypt?



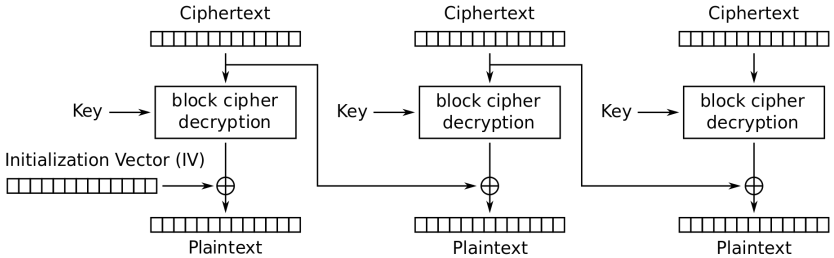
Cipher Block Chaining (CBC) mode encryption

CBC Mode: Decryption

- How do we decrypt CBC mode?
  - Parse ciphertext as (IV, C_1, …, C_m)
  - Decrypt each ciphertext and then XOR with IV or previous ciphertext



Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode decryption

CBC Mode: Decryption

$$C_i = E_K(M_i \oplus C_{i-1})$$ Definition of encryption

$$D_K(C_i) = D_K(E_K(M_i \oplus C_{i-1}))$$ Decrypting both sides

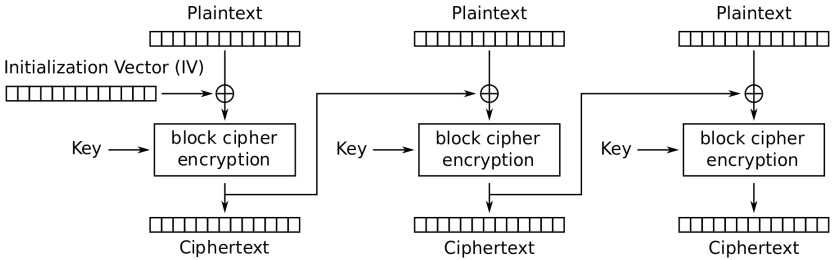
$$D_K(C_i) = M_i \oplus C_{i-1}$$ Decryption and encryption cancel

$$D_K(C_i) \oplus C_{i-1} = M_i \oplus C_{i-1} \oplus C_{i-1}$$ XOR both sides with C_{i-1}

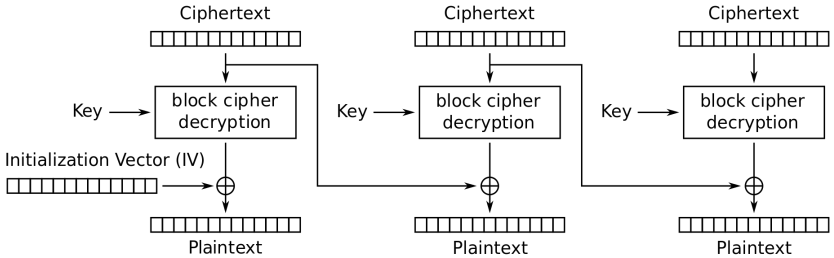
$$D_K(C_i) \oplus C_{i-1} = M_i$$ XOR property

CBC Mode: Efficiency & Parallelism

- Can encryption be parallelized?
  - No, we have to wait for block i to finish before encrypting block i+1
- Can decryption be parallelized?
  - Yes, decryption only requires ciphertext as input



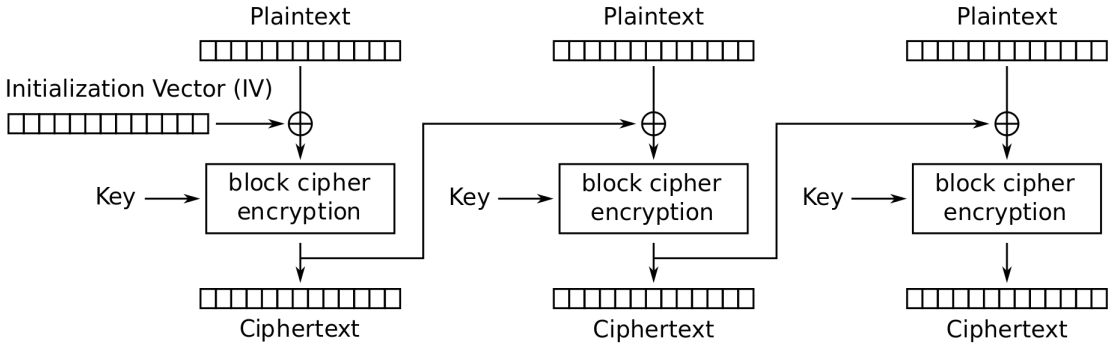
Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode decryption

CBC Mode: Padding

- What if you want to encrypt a message that isn't a multiple of the block size?
  - AES-CBC is only defined if the plaintext length is a multiple of the block size
- Solution: Pad the message until it's a multiple of the block size
  - **Padding**: Adding dummy bytes at the end of the message until it's the proper length



Cipher Block Chaining (CBC) mode encryption

CBC Mode: Padding

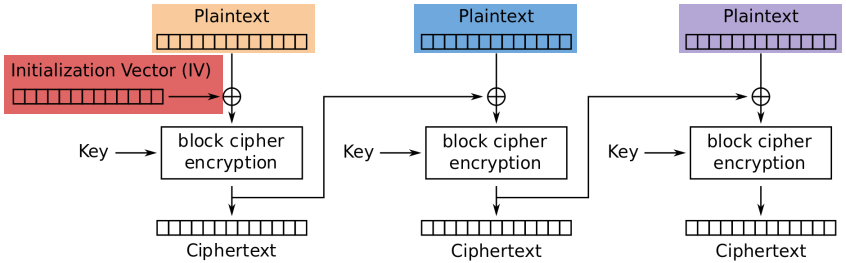
- What padding scheme should we use?
  - Padding with 0's?
    - Doesn't work: What if our message already ends with 0's?
  - Padding with 1's?
    - Same problem
- We need a scheme that can be unpadded without ambiguity
  - One scheme that works: Append a 1, then pad with 0's
    - If plaintext is multiple of n, you still need to pad with an entire block
  - Another scheme: Pad with the number of padding bytes
    - So if you need 1 byte, pad with **01**; if you need 3 bytes, pad with **03 03 03**
    - If you need 0 padding bytes, pad an entire dummy block
    - This is called PKCS #7
    - If no padding was needed, PKCS #7 would still add a full block of padding (e.g., 16 bytes of 0x10 if block size is 16), so it's never ambiguous.

CBC Mode: Security

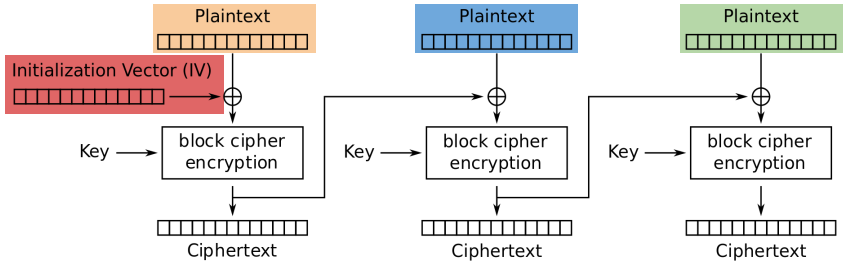
- AES-CBC is IND-CPA secure. With what assumption?
  - The IV must be randomly generated and never reused
- What happens if you reuse the IV?
  - The scheme becomes deterministic: No more IND-CPA security

CBC Mode: IV Reuse

- Consider two three-block messages: $P_1P_2P_3$ and $P_1P_2P_4$
  - The first two blocks are the same for both messages, but the last block is different
  - What if we encrypt them with the same IV?
- When the IV is reused, CBC mode reveals when two messages start with the same plaintext blocks, up to the first different plaintext block



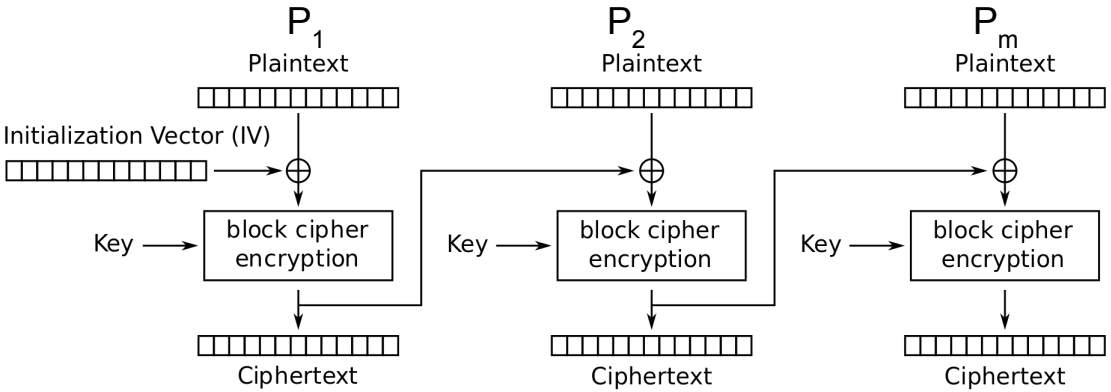
Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode encryption

CBC Mode is IND-CPA (when used correctly)

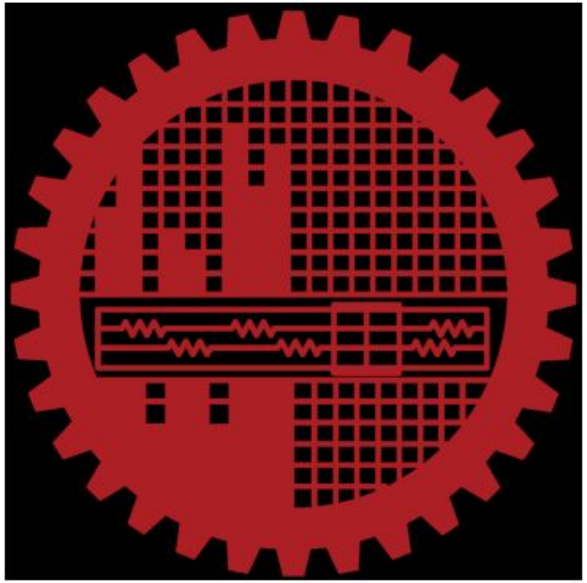
- Enc(K, M):
  - Split M in m plaintext blocks P_1 … P_m each of size n
  - Choose random IV, compute and output (IV, C_1, …, C_m) as the overall ciphertext
- Why IND-CPA?
  - If there exists an attacker that wins in the IND-CPA game, then there exists an attacker that breaks the block cipher security. Proof is out of scope.



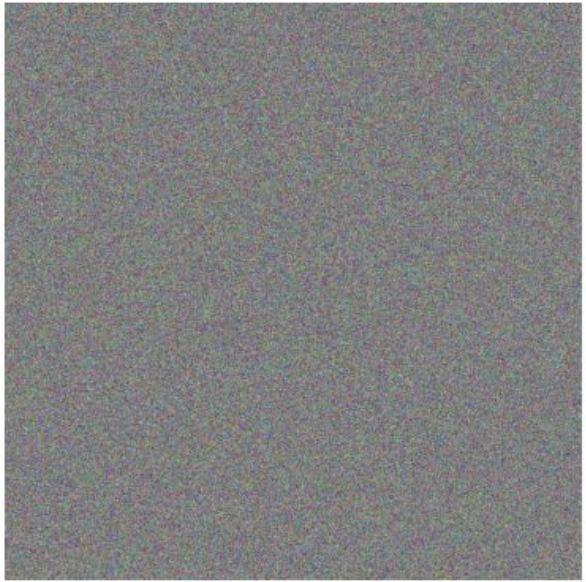
Cipher Block Chaining (CBC) mode encryption

CBC Mode: IND-CPA Secure

Original



Encrypted (CBC)



Decrypted

